4. In-Memory Data Management

Operating Systems (OS)
Master in Innovation and Research in Informatics (MIRI)
Facultat d'Informàtica de Barcelona (FIB)
Universitat Politècnica de Catalunya (UPC)
2025/2026 Q1

FIB

READING REPORT

[Terry13] Terry, D., *Replicated data consistency explained through baseball*. Communications of the ACM, Volume 56, Issue 12, pp. 82-89, 2013.

Contents

- Introduction
- History
- Eventual consistency
- Examples

Contents

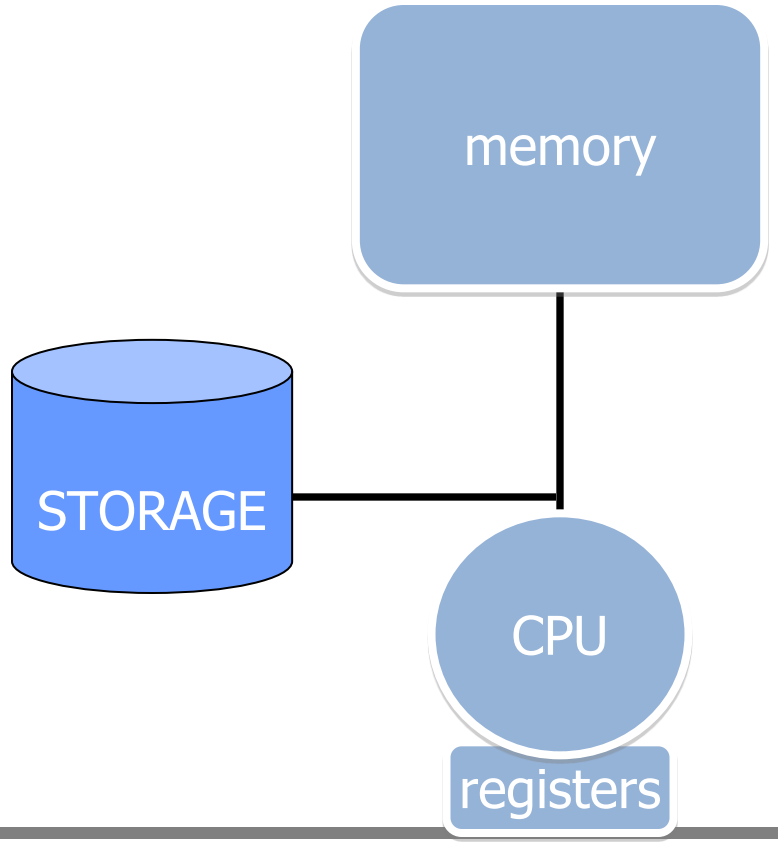
- **Introduction**
- History
- Eventual consistency
- Examples

Introduction

- What is Data Management?
  - As an operating system …
  - **Data** is any information useful for your users.
  - **Management**:
    - Where do we save this data?
    - How do we access this data?

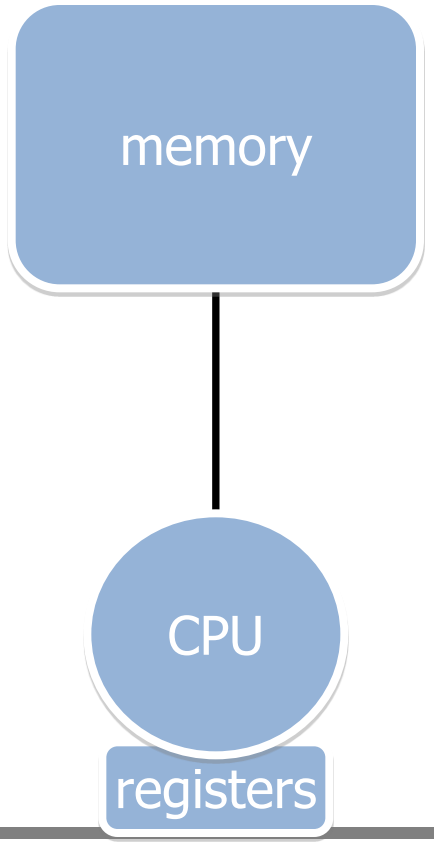
Introduction

- How do we manage data in a Desktop Computer?
  - Two options:
    - Memory
    - Secondary storage



Introduction

- How do we manage data in a Desktop Computer?
  - Two options:
    - Memory ← **Focus on this**
    - Secondary storage
- And in a WSC?
  - Evolution
  - Concepts
  - Problems and specific solutions



Contents

- Introduction
- **History**
- Eventual consistency
- Examples

History

- Desktop Computer
- Pre “WSC”
- WSC

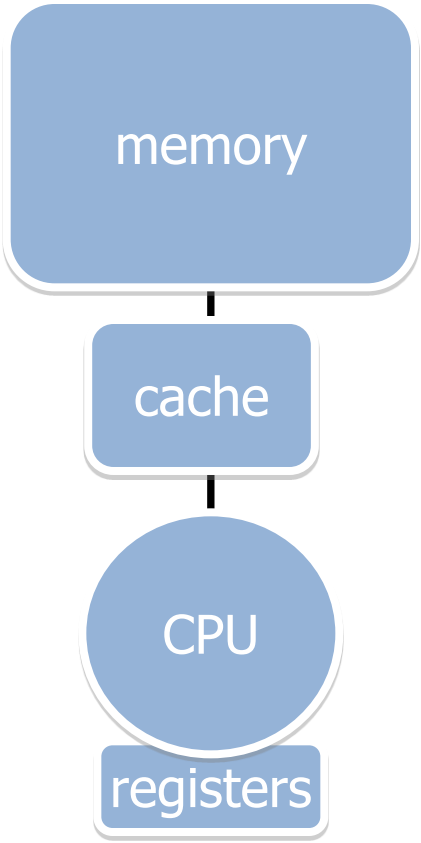
Memory

- Memory is a direct-access large array of bytes
- Each byte can be referenced from a program through its own address
  →The address space
- Address is just a number 0..2^N
- Memory is accessed with specific instructions
  - Load
  - Store
- Completing a memory access may take many cycles of the CPU clock
  - ~83 ns (250 cycles) [Duar08]

Admiral Grace Hopper explains Nanoseconds (https://www.youtube.com/watch?v=9eyFDBPk4Yw)

Cache memory

- Add **cache** to speed up memory access without any OS intervention
  - CPU reads from cache instead of RAM
    - L1: ~1 ns (3 cycles) [Duar08]
    - L2: ~4.7 ns (14 cycles) [Duar08]
    - RAM: ~83 ns (250 cycles) [Duar08]
  - Cache keeps last accessed data
    - Exploit temporal locality
  - **Entire cache lines** loaded from RAM
    - Exploit spatial locality
- No need to change programs
    - HW does all the work



Cache memory

- Example:

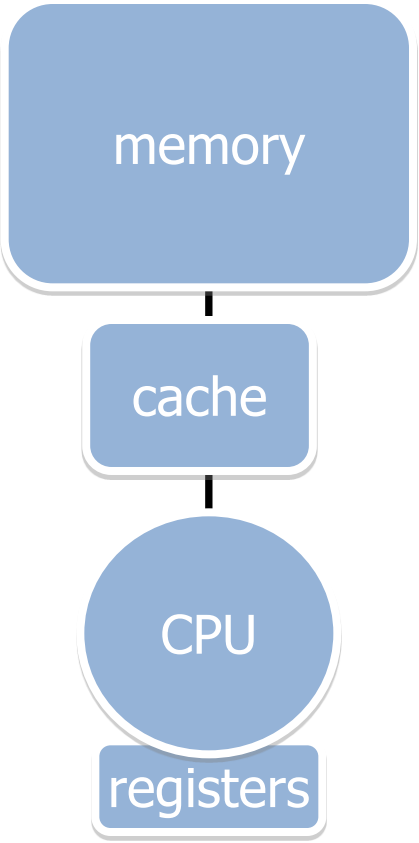
  LD 1000, REG

  LD 1000, REG

  ST 666, 1000

  LD 1000, REG

- What happens to our architecture?
  https://forms.gle/p3p425ejgaaVKZMA7

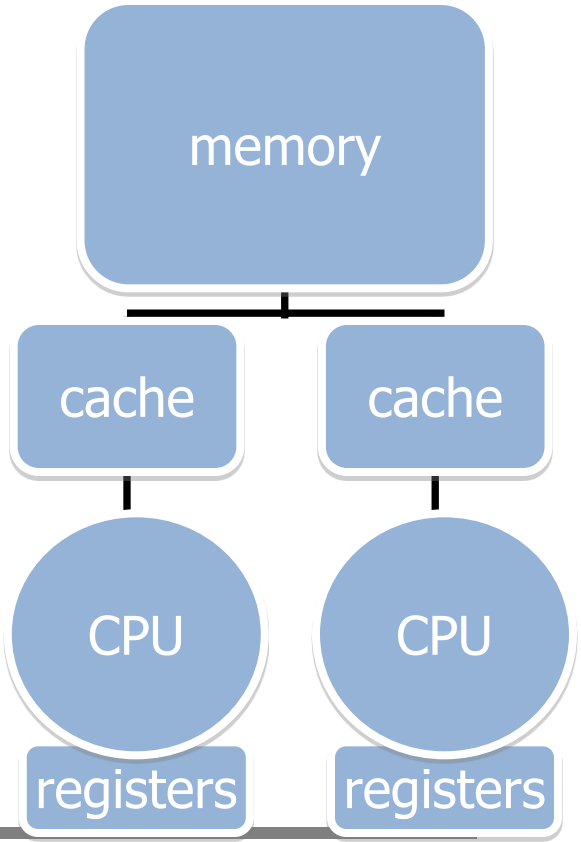


Memory: Desktop Computer

- **Add multiple processors**
- Keep same memory…
  - **Same address space (Shared)**
  - Replicate previous idea
    - Add caches
- First problem arise
  - Both CPUs access same @
  - How do you maintain consistency?
    - **Coherence protocols**

[FUNK16] https://scratch.mit.edu/projects/112080222/

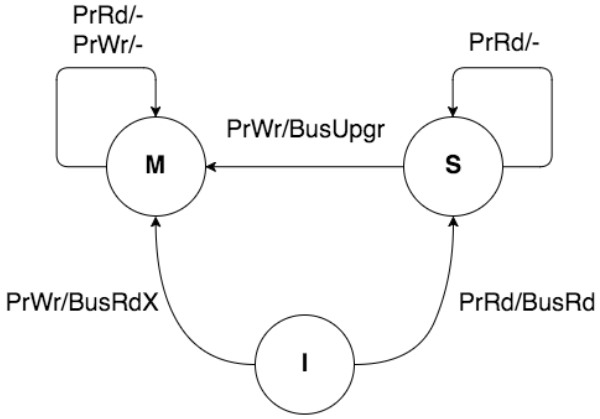
- What happens to our architecture?
  https://forms.gle/A6T9q8XHkVoBmAuT7



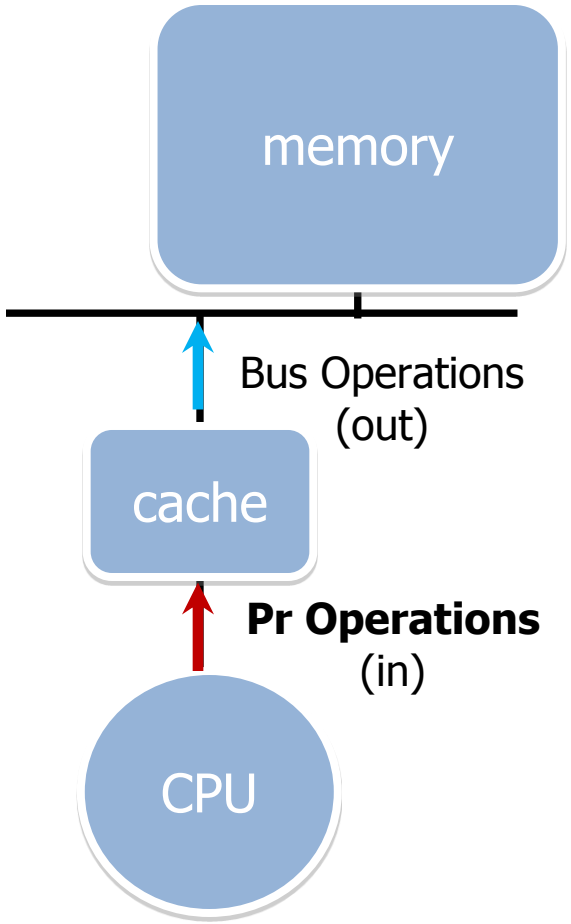
Memory: Desktop Computer

- Coherence protocol
  - Finite State Machine
    - guided by **Processor** and **Bus** operations
  - Define each cache line State
- Different protocols
  - Simplest: MSI
    - **M**odified
    - **S**hared
    - **I**nvalid
- Snooping the memory bus for operations
  - Monitor ALL the ops!

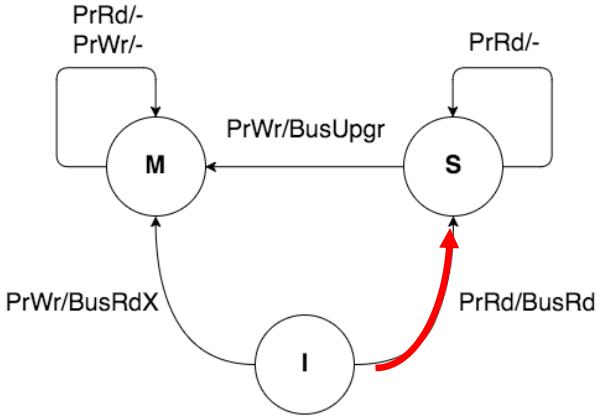
Memory: Desktop Computer



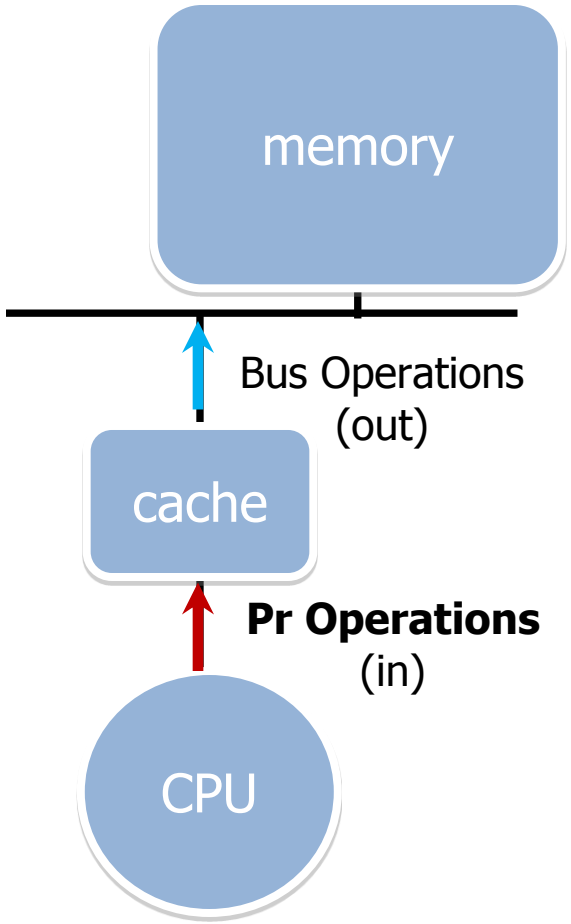
Courtesy of [Sand16P]



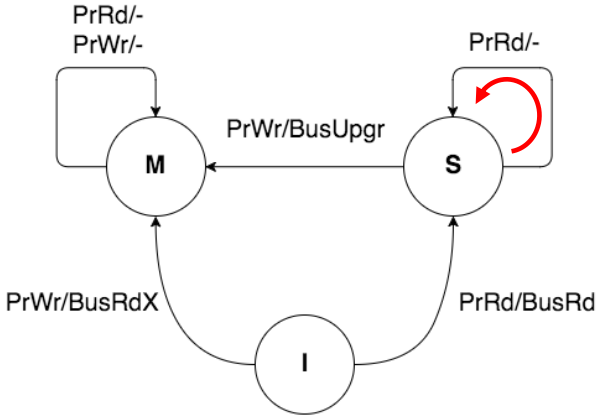
Memory: Desktop Computer



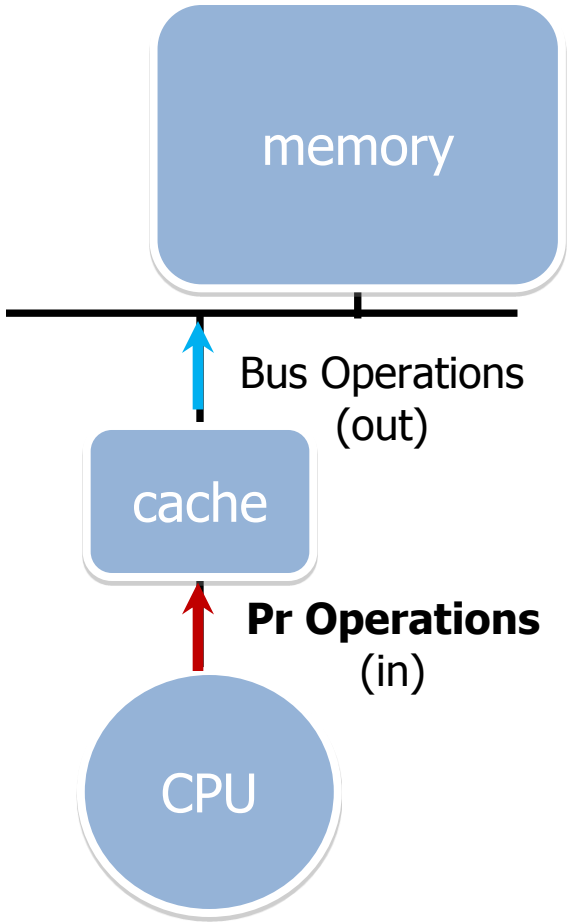
Courtesy of [Sand16P]



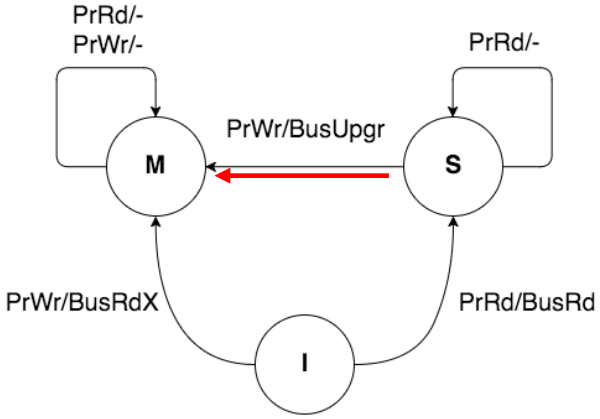
Memory: Desktop Computer



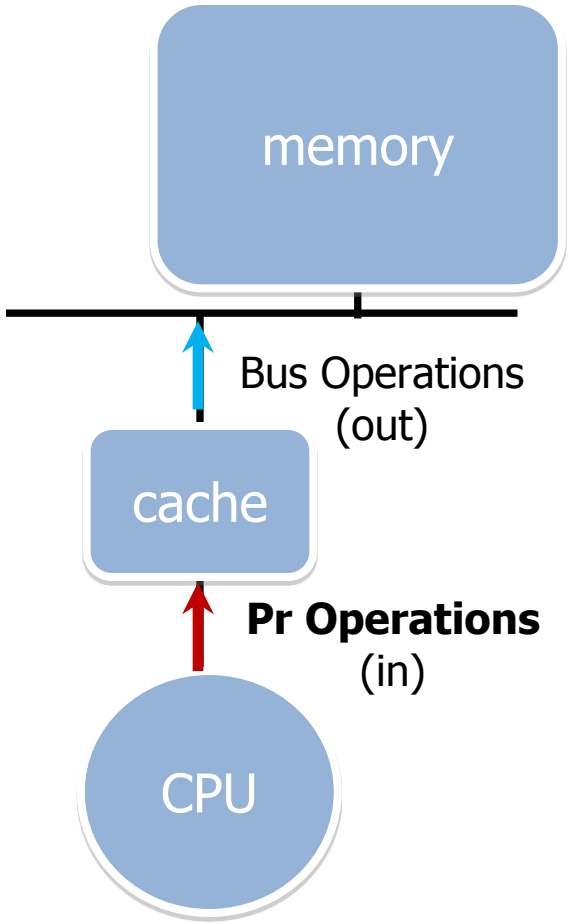
Courtesy of [Sand16P]



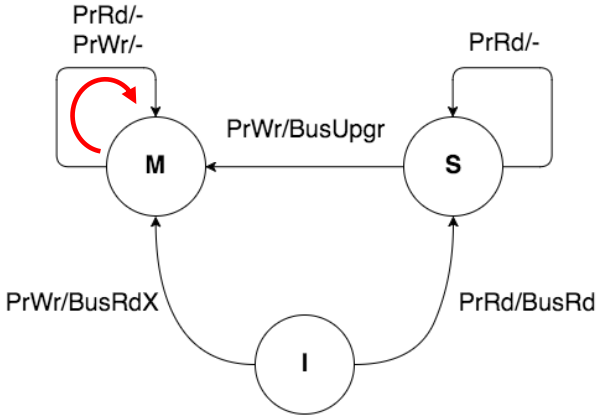
Memory: Desktop Computer



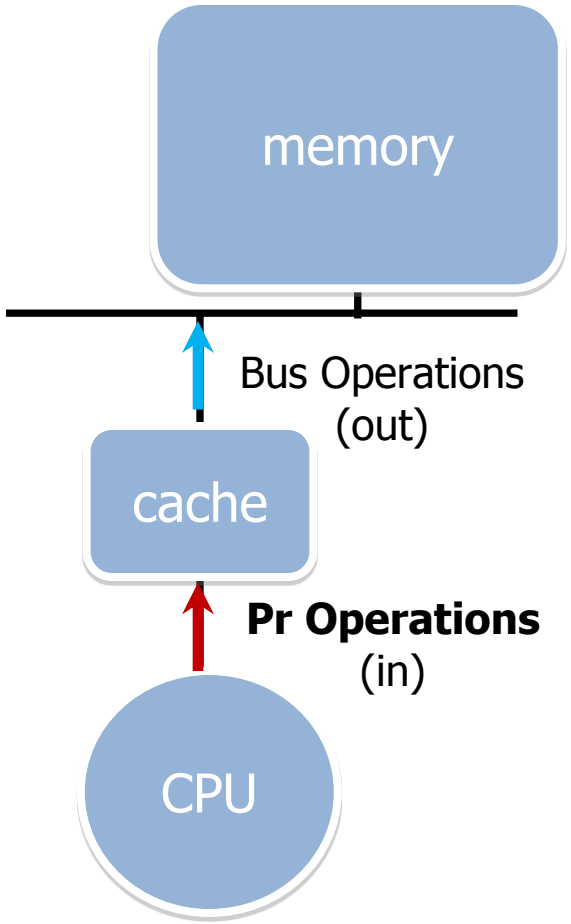
Courtesy of [Sand16P]



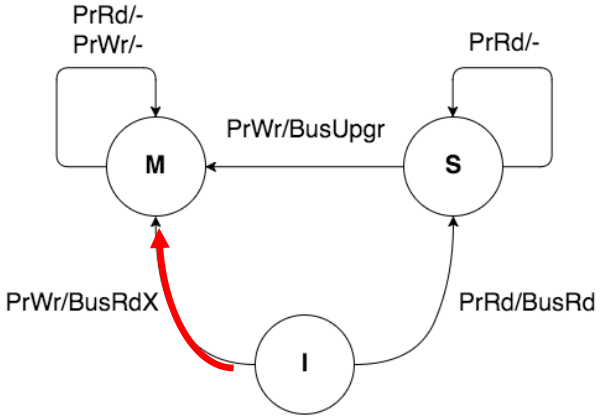
Memory: Desktop Computer



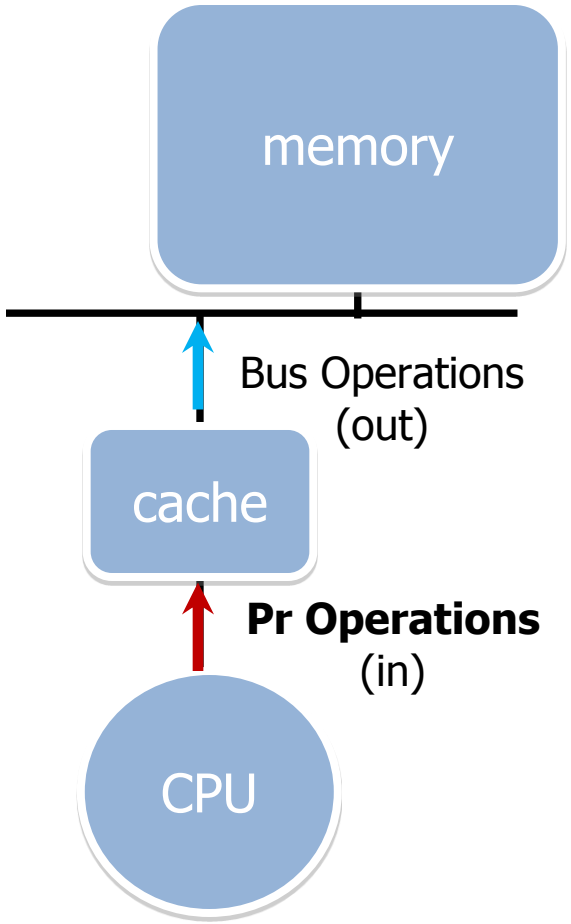
Courtesy of [Sand16P]



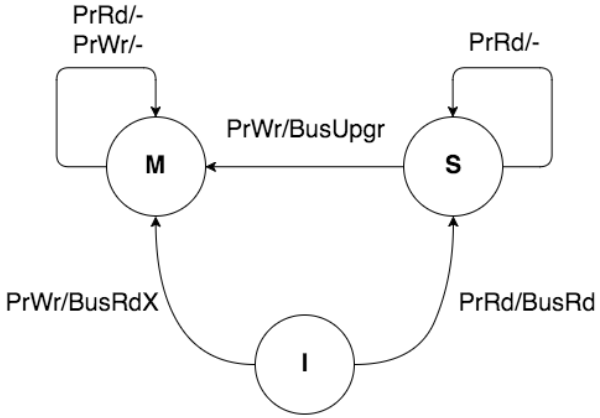
Memory: Desktop Computer

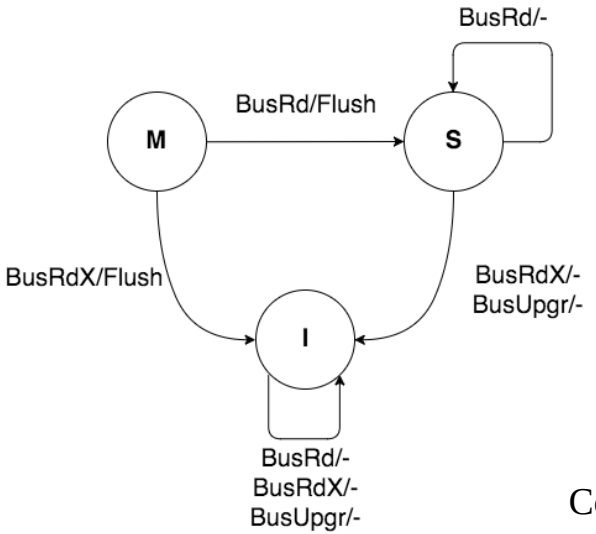


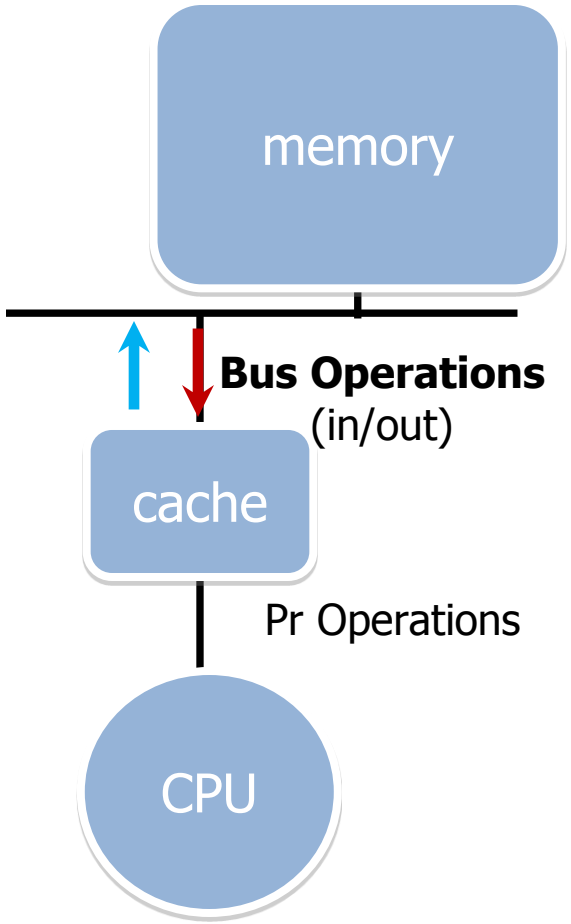
Courtesy of [Sand16P]



Memory: Desktop Computer





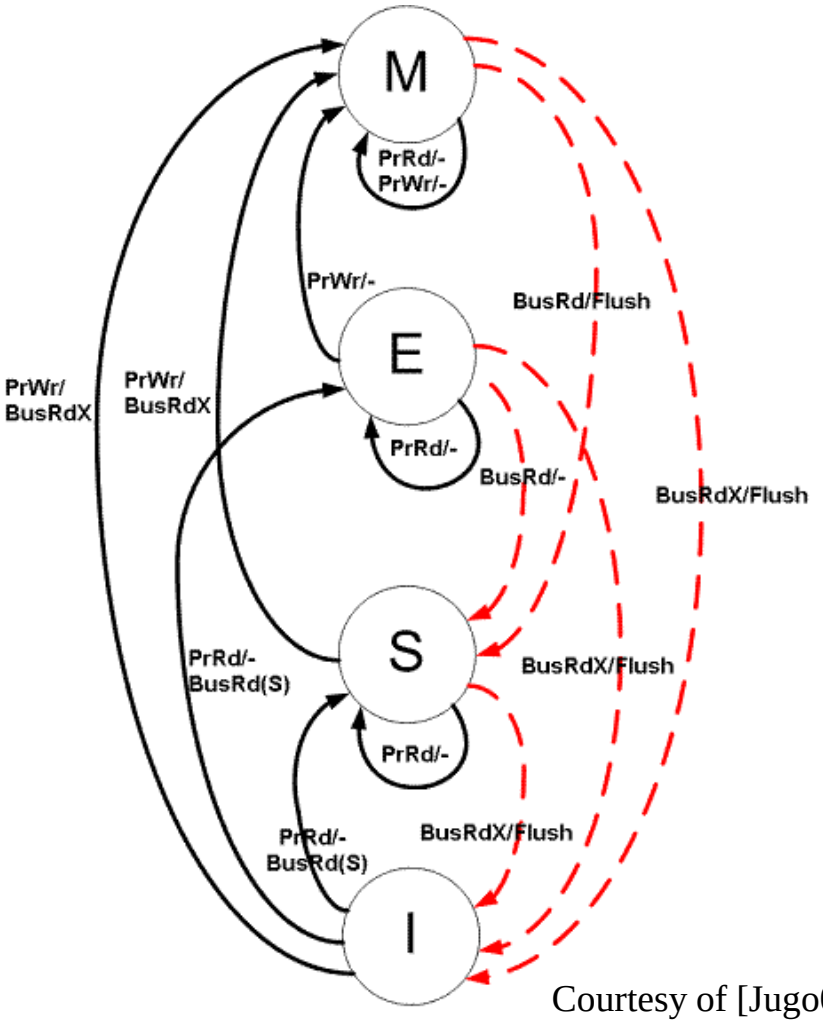


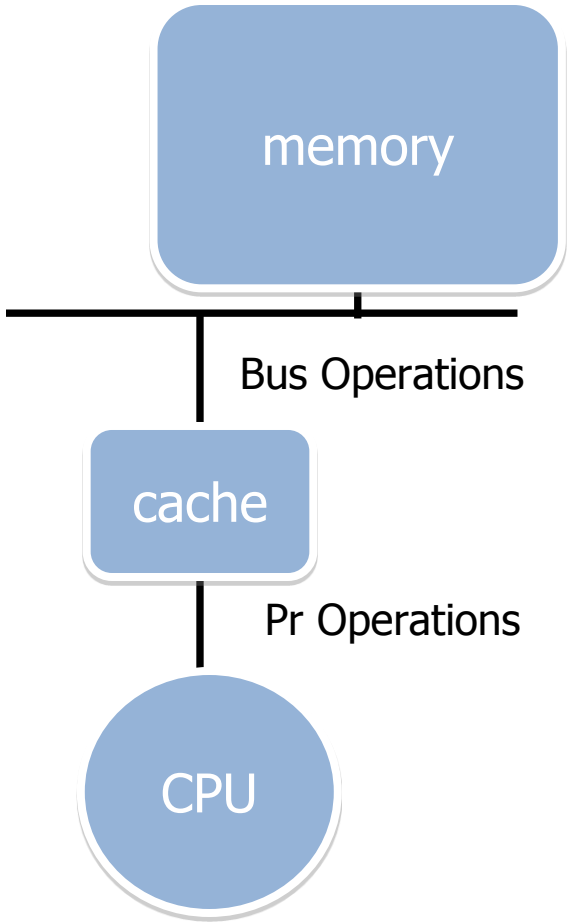
Courtesy of [Sand16B]

Memory: Desktop Computer

- Other Optimizations
  - Usually try to avoid accesses to main memory
  - Favor the cache-to-cache transfers
- MESI
    - **E**xclusive – **Read** cache line is owned by 1 cache
- MOESI
    - **O**wned – **Modified** cache line may be SHARED by others
      - Delays writes to main memory
- MOESIF
    - **F**orward – Shared cache line may be forwarded to others
      - Avoid broadcast in the bus

Memory: Desktop Computer





Courtesy of [Jugo09]

https://www.scss.tcd.ie/Jeremy.Jones/vivio/caches/MESI.htm

Desktop Computer Summary

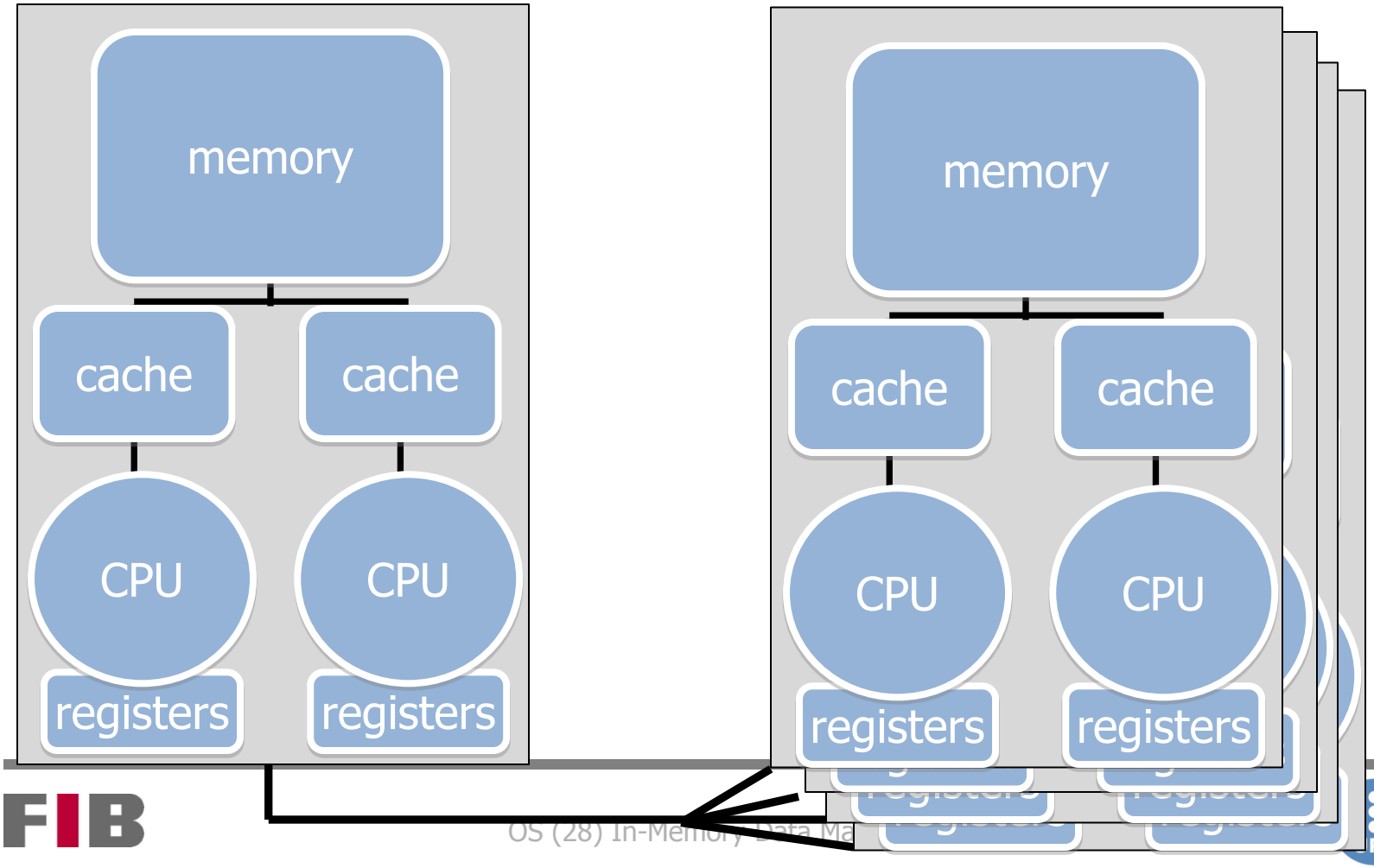
- Single node: CPU + memory
- Memory access is slow
  - → Add cache to CPU
- Multiple CPUs → Multiple caches
  - → Coherence protocol to maintain consistency

History

- Desktop Computer
- **Pre "WSC"**
- WSC

Memory: "Pre" WSC

- N nodes interconnected through a network



Memory: "Pre" WSC

- N nodes, multiple processors
  - N is small (2 ~ 100nodes)
  - Each node has its own memory chip
    - MULTIPLE address spaces!
  - Local-Area Network
- What are the options to access data?
  - Direct view of HW resources → **distributed** address space
    - Each node has its own private address space
      - M1, M2, …, MN
  - Single address space → **shared** address space
    1) A single @ space is *replicated* through all nodes
       - M1, M1, …, M1
    2) An *aggregation* of all private @ spaces
       - M1 + M2 + … + MN

Memory: "Pre" WSC

- **Distributed address space**
  - The @ space of other nodes are not accessible
  - Message passing mechanisms are needed
  - Explicit communication is required to programmer
  - Example: Message Passing Interface (MPI)

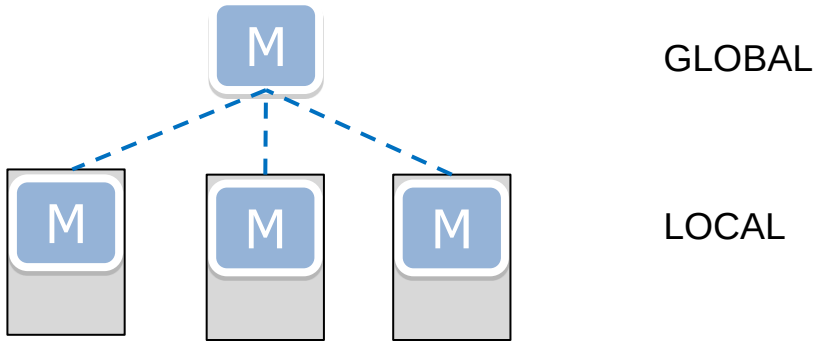
WE WILL NOT CONSIDER THIS CASE

Distribute Data is hard, and users want a shared access anyway

(and this case does NOT need any middleware ;)

Memory: "Pre" WSC

- **Shared address space**
  - **Add specific layer**(SW/HW) offering this @space
    - Software: DSM
    - Hardware: NUMA
- 1) *Replicated @ space*
  - Each node has a replica of the @ space



→ like a cache

Implement a replicated @ space

- Same problems as in a multiprocessor
  - How maintain the values in @ space consistent?
- How do *processor* detects memory accesses?
  - Specific instructions: Load & Store
    - Generates Bus Operations
  - Loaded into cache (keep state)
- How do *middleware* detect memory access?
  - Virtual Memory!
    - Pages (4096B)
    - Page tables
      - Memory protection: R, W, None
      - Page Fault Exception!

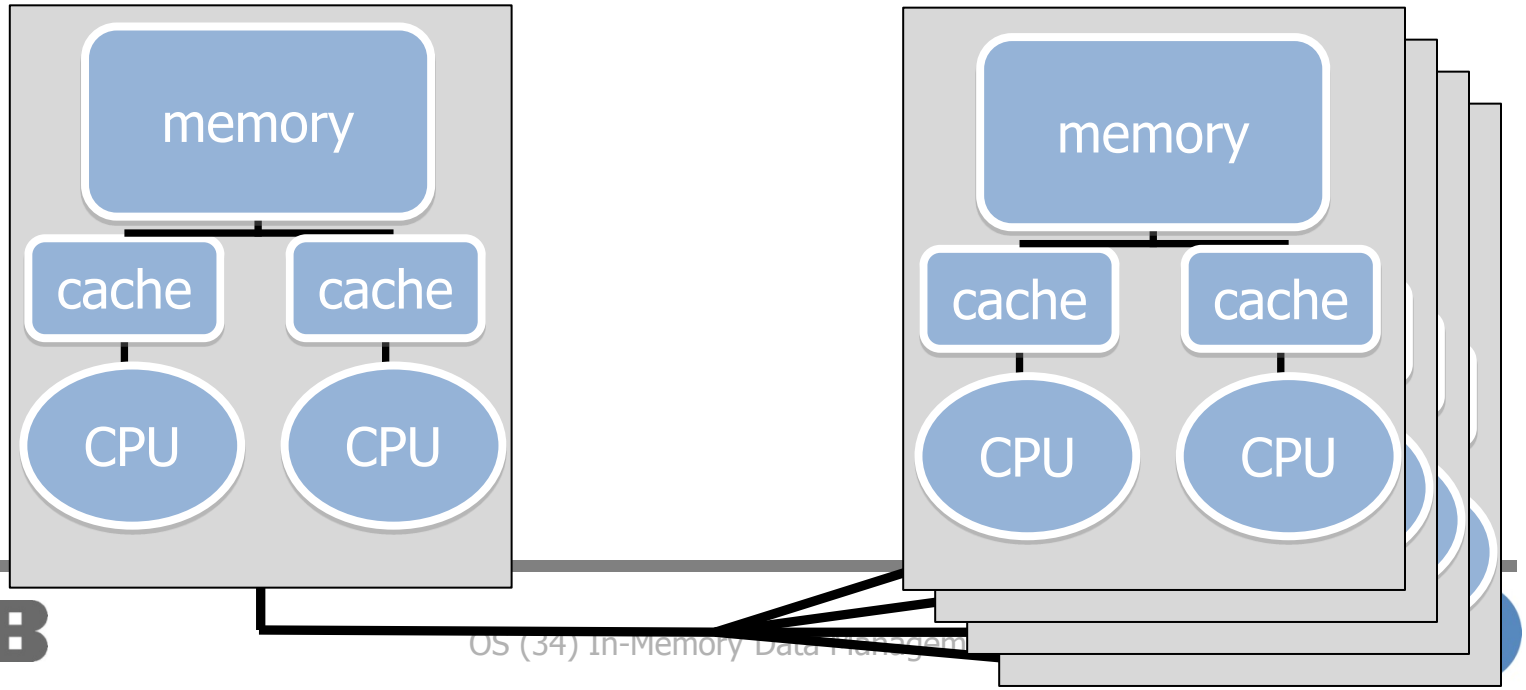
Shared address space: *Replicated @ space*

- *Directory* to keep page state in global @space

- And each node also has local caches!!
  - So, there are *two levels* of replicas
  - An additional level into the memory hierarchy

The problem with replicas

- What's the problem with replicas?
- What should happen when a cpu…
  - … reads a value?
  - … writes a value?



The problem with replicas

- Replicas → Consistency problem
  - How do we maintain the data consistent?
- https://scratch.mit.edu/projects/112522130

The problem with replicas

- Replicas → Consistency problem
  - How do we maintain the data consistent?
- https://scratch.mit.edu/projects/112522130
- But now… the bus is NOT present!!
  - (or at least it has a higher latency)
  - Usually some kind of network

The problem with replicas

- Replicas → Consistency problem
  - How do we maintain the data consistent?
- https://scratch.mit.edu/projects/112522130
- But now… the bus is NOT present!!
  - (or at least it has a higher latency)
  - Usually some kind of network
- Let's think about how to implement different consistencies…

Consistency models

- Strong consistency
  - All replicas updated before a write completes.
  - Problems:
    - What's the latency of a write?
    - What happens to local caches?
    - What's the *cache line size* now?
    - What happens when two processors write two different addresses that fits in the same **cache line**?
      - False sharing
  - Main problem: emulate memory bus
  - Do we really need this strong consistency?

Consistency models

- Relax consistency: Treadmarks❖
  - Treadmarks is a software DSM
  - Allow writes on different memory regions
  - Allow writes on same memory region
    → calculate diffs & merge changes
  - But programmer needs to mark all critical regions
  - Based on locks (Acquire & Release)
    - Therefore, it groups a set of adresses and operations
  - Lazy Release Consistency model

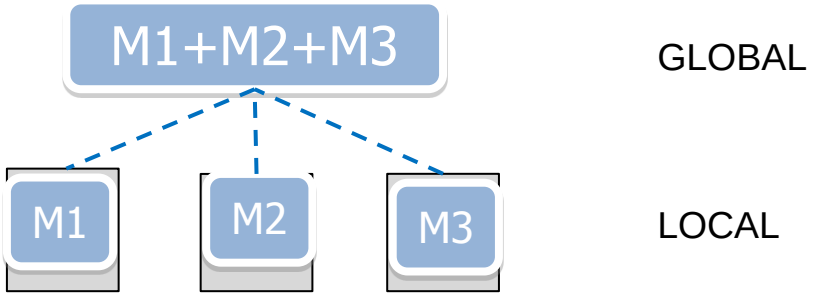
❖ *TreadMarks: Distributed Shared Memory on Standard Workstations and Operating Systems.* Keleher et al. https://www.usenix.org/legacy/publications/library/proceedings/sf94/full_papers/keleher.a

Consistency models: Summary

- Consistency model changes the memory access granularity → not fixed size accesses but dynamic (a set of addresses plus some operations)
- This granularity will keep evolving to bigger data structures or even objects with its related operations
- This coarse access loses the notion of @space and enables reasoning about higher level abstractions

Memory: "Pre" WSC

- **Shared address space (Continuation)**
- 1) *Replicated @ space*
- 2) *Aggregation of @ spaces*



  - Large @ space
  - Each node is responsible for a part of the @ space
  - No replicas → No consistency problems
  - But…
    - No replicas → Access to @ spaces from other nodes → Latency

Summary: Memory: "Pre" WSC

- Multiple nodes
- Different address spaces
  - Distributed
  - Shared
    - Replicated
    - Agregated
  - A Middleware is required to solve consistency
- Relax consistency → change granularity access

Memory: "Pre" WSC

- Main problems…
  - Till now we were thinking in a perfect world, but…
  - What happens when a network error occurs and a node gets isolated from the others?
    - » This is called a *Network Partition*
  - Consistency can not be maintained any more
  - Data may not be available

History

- Desktop Computer
- Pre “WSC”
- **WSC**

Memory: WSC

- Scale up the number of nodes
  - 1000, 10^6, …
  - Wide-Area Network!
- What happens to our shared cases?
  1) *Replicated>*
     - Latency of writes is so high that is not viable anymore.
       - No Availability (A)
     - Think on the number of invalidation´s network messages that must be sent for a single write → Network traffic
     - Performance simply disappears
     - Partitions(P) are NOT tolerated
     - Only focused on maintaining the Consistency (C)

Memory: WSC

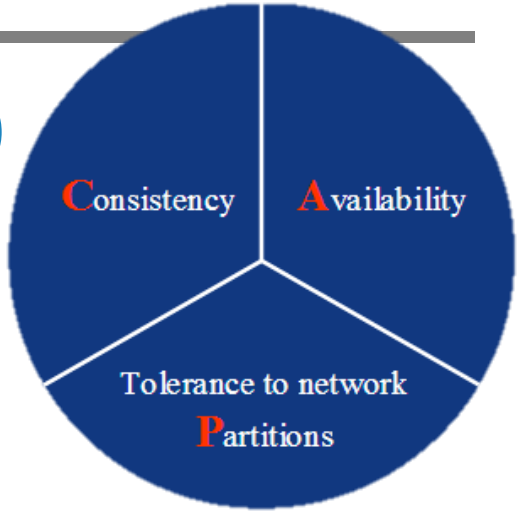
- What happens to our shared cases?
  - 2) *Aggregated @ space>*
    - Similar problems
    - High latency
    - No Partition tolerance
    - No availability

Contents

- Introduction
- History
- **Eventual consistency**
- Examples

Memory: "Pre" WSC

- CAP theorem (Eric Brewer 2000)
  - **C**onsistency
  - **A**vailability
  - (Tolerate) **P**artition



- You can have at most 2 of these properties:
  - Consistency + Availability → No tolerate partition
  - Consistency + Tolerate Partition → No availability
  - Availability + Tolerate Partition → No consistency

Eric Brewer. *Towards Robust Distributed Systems*
http://www.eecs.berkeley.edu/~brewer/cs262b-2004/PODC-keynote.pdf

Memory: WSC

- Following CAP theorem, If
  - data must be available
  - and system must tolerate network partitions

Memory: WSC

- Following CAP theorem, If
  - data must be available
  - and system must tolerate network partitions

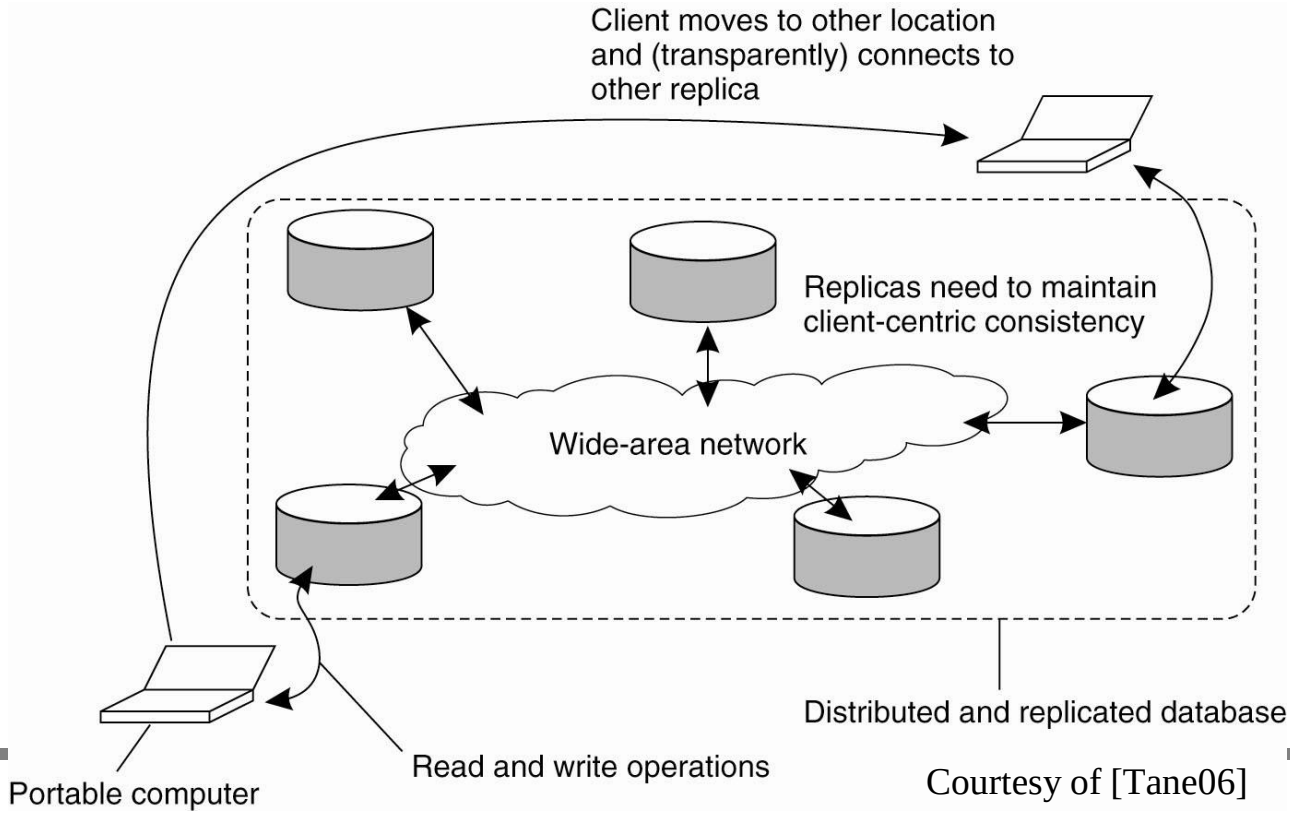
  →**Relax consistency**
- Eventual consistency
  - After a set of updates, the data will be consistent across the whole system *EVENTUALLY*
  - Writes are fast
    - Local only (but propagated in the background)
  - Reads are fast too
    - …but may be rotten

Eventual Consistency

- What does "Eventually" mean?
  - **Inconsistency Window**
    - The period between the update and the moment when it is guaranteed that any observer will always see the updated value
    - (It also applies to the other relaxed models)
  - It depends on the inner *implementation*, latency, network partitions, …
    - … any problem that MAY happen
  - But, without problems…
    - https://gist.github.com/jboner/2841832

Eventual Consistency

- Works well IFF every client always updates the **same** replica
  - if clients are mobile…



Courtesy of [Tane06]

Eventual Consistency

- In order to limit the 'inconsistency window' data stores offer some *guarantees*
- Usually the consistency models are **data-centric**:
  - Consistent prefix
  - Bounded staleness
- But in some environments it is better to apply a **user-centric** vision:
  - Monotonic reads
  - Read your writes

Guarantees

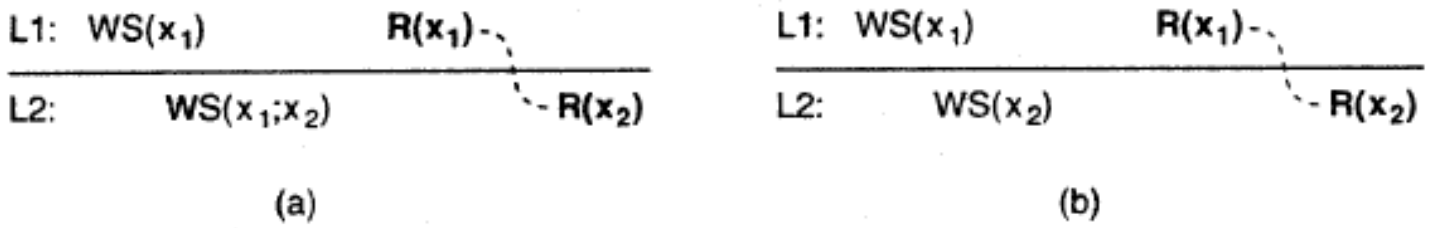
- Monotonic reads
- Consistent prefix
- Read your writes
- Bounded staleness

Guarantees

- **Monotonic reads**
- Consistent prefix
- Read your writes
- Bounded staleness

Monotonic Read

- If process P reads a data item *X* on server A, any successive read operation on *X* by P on another server will always return that same value or a more recent value



Courtesy of [Tane06]

L1: Replica 1
L2: Replica 2

Monotonic Read

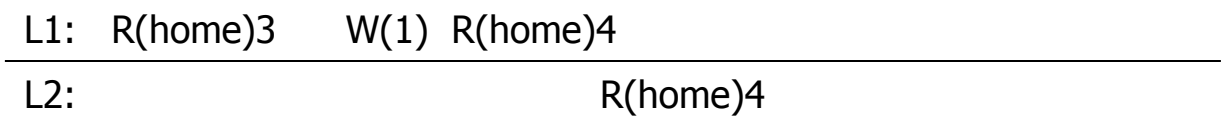
- Example: [Terry13]
  - Current score: 2-3 (visitors=2 and home=3)
  - Updates:
    - (1) home = 4
    - (2) home = 5
  - Valid scores: 2-3, 2-4, 2-5

L1: R(home)3 W(1)

L2:

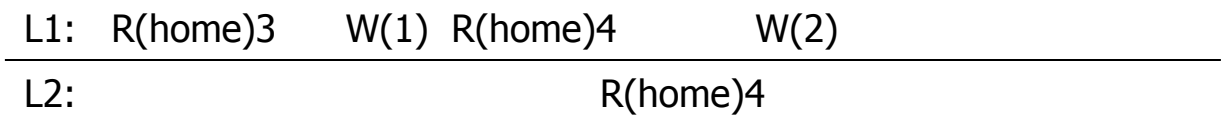
Monotonic Read

- Example: [Terry13]
  - Current score: 2-3 (visitors=2 and home=3)
  - Updates:
    - (1) home = 4
    - (2) home = 5
  - Valid scores: 2-3, 2-4, 2-5



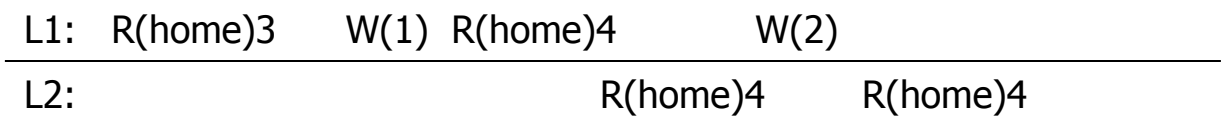
Monotonic Read

- Example: [Terry13]
  - Current score: 2-3 (visitors=2 and home=3)
  - Updates:
    - (1) home = 4
    - (2) home = 5
  - Valid scores: 2-3, 2-4, 2-5



Monotonic Read

- Example: [Terry13]
  - Current score: 2-3 (visitors=2 and home=3)
  - Updates:
    - (1) home = 4
    - (2) home = 5
  - Valid scores: 2-3, 2-4, 2-5



Monotonic Read

- Example: [Terry13]
  - Current score: 2-3 (visitors=2 and home=3)
  - Updates:
    - (1) home = 4
    - (2) home = 5
    - (3) visitors = 3
  - Valid scores: ???

L1:	R(home)3	W(1)	W(2)	W(3)
L2:				

Guarantees

- Monotonic reads
- **Consistent prefix**
- Read your writes
- Bounded staleness

Consistent prefix

- A reader is guaranteed to observe an ordered sequence of writes starting with the first write to the data store.

Consistent prefix

- Example: [Terry13]
  - Current score: 0-0 (visitors=0 and home=0)
  - Updates:
    - (1) home = 1
    - (2) visitors = 1
    - (3) home = 2
  - Valid scores: ???

L1: R(home)0 W(1)

L2:

Consistent prefix

- Example: [Terry13]
  - Current score: 0-0 (visitors=0 and home=0)
  - Updates:
    - (1) home = 1
    - (2) visitors = 1
    - (3) home = 2
  - Valid scores: ???

L1:	R(home)0	W(1)	W(2)	W(3)
L2:				

Guarantees

- Monotonic reads
- Consistent prefix
- **Read your writes**
- Bounded staleness

Read your writes

- The effect of a write operation by a process on data item X will always be seen by a successive read operation on X by the same process.

Read your writes

- Example: [Terry13]
  - Current score: 2-3 (visitors=2 and home=3)
  - Updates:
    - (1) home = 4
    - (2) home = 5
  - Valid scores: ???

L1:	R(home)3		W(1)	R(home)4			W(2)		
L2:						R(home)4		R(home)	
L3:								R'(home)	

Guarantees

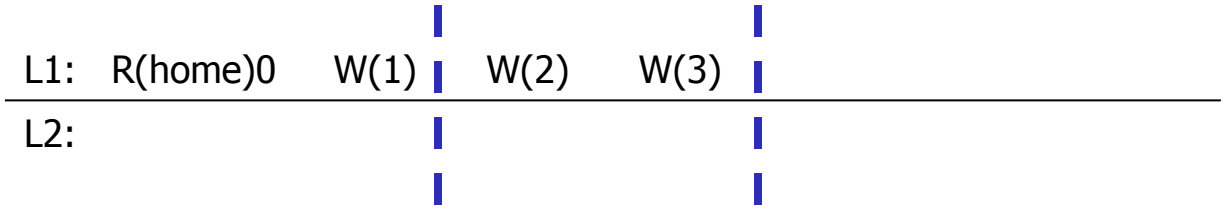
- Monotonic reads
- Consistent prefix
- Read your writes
- **Bounded staleness**

Bounded staleness

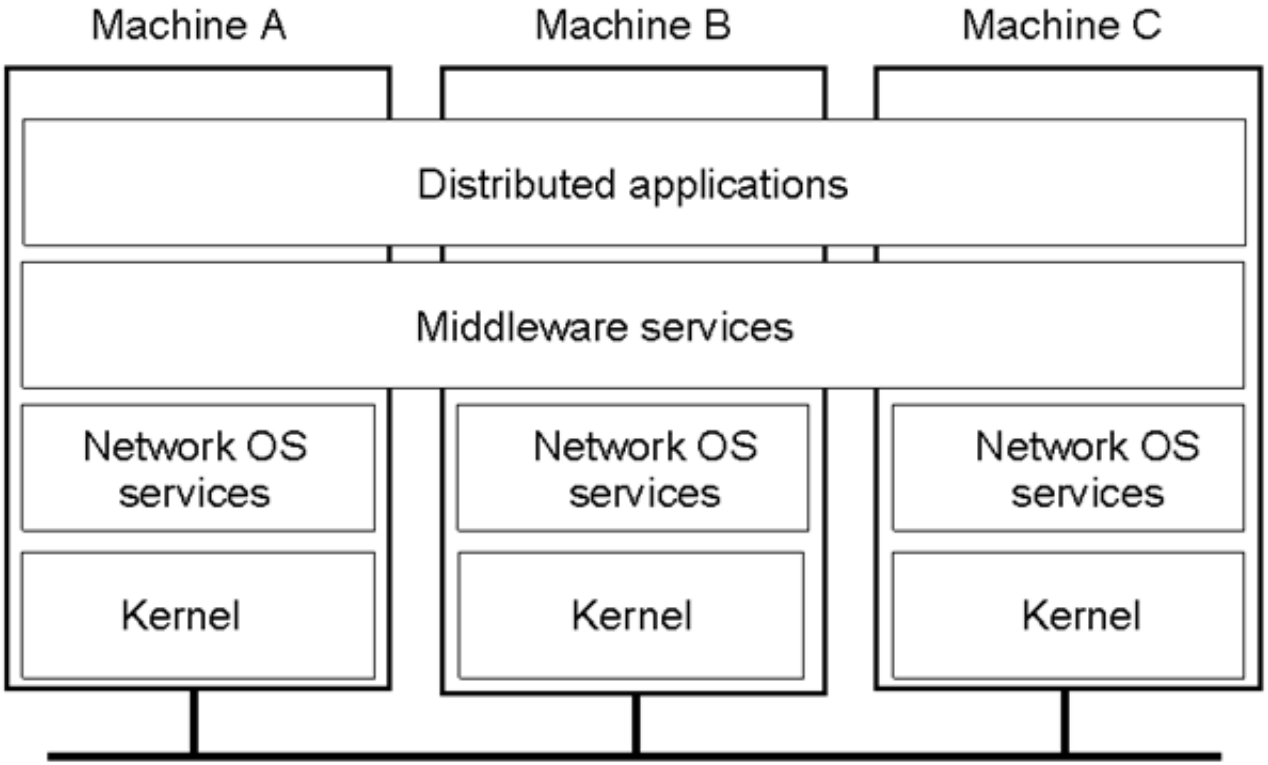
- The storage system guarantees a read operation will return any values written more than *T* minutes ago or more recently written values.

Bounded staleness

- Example: [Terry13]
  - Current score: 0-0 (visitors=0 and home=0)
  - Updates:
    - (1) home = 1
    - (2) visitors = 1
    - (3) home = 2
  - Valid scores: ???



WSC Architecture



- How do you enforce any of these consistencies?

Consistency protocols

- Consistency protocols describe:
  A. Implementations of specific consistency models
     - We focus on strong and eventual consistency
  B. Architectures of replicated processes for fault tolerance
     - Hierarchical and flat process groups
- Consistency protocols discussed:
  1. Primary-based protocols
     - Writes go to a single replica
  2. Replicated-write protocols
     - Writes can go to any replica

Contents

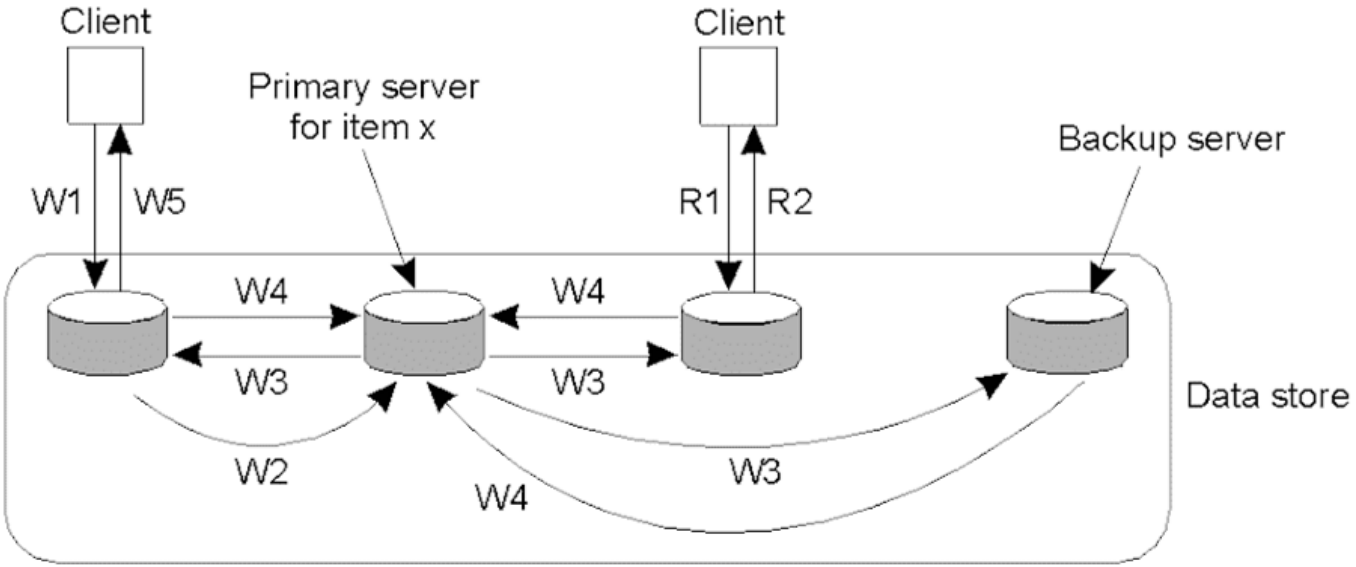
- Consistency protocols
  - **Primary-based protocols**
  - Replicated-write protocols

Primary-based protocols

- Each data item is associated with a **primary replica** which is in charge of coordinating write operations to the data item
  - If the primary fails, one of the replicas is elected to act as the primary
- Also called passive replication protocols
- Two types:
  A. Primary-backup remote-write protocols
  B. Primary-backup local-write protocols

Primary-backup remote-write protocols

- All writes are done at a fixed single replica
- Reads can be carried out locally



W1. Write request
W2. Forward request to primary
W3. Tell backups to update
W4. Acknowledge update
W5. Acknowledge write completed

R1. Read request
R2. Response to read

Courtesy of [Tane06]

Primary-backup remote-write protocols

↓ Bad write performance

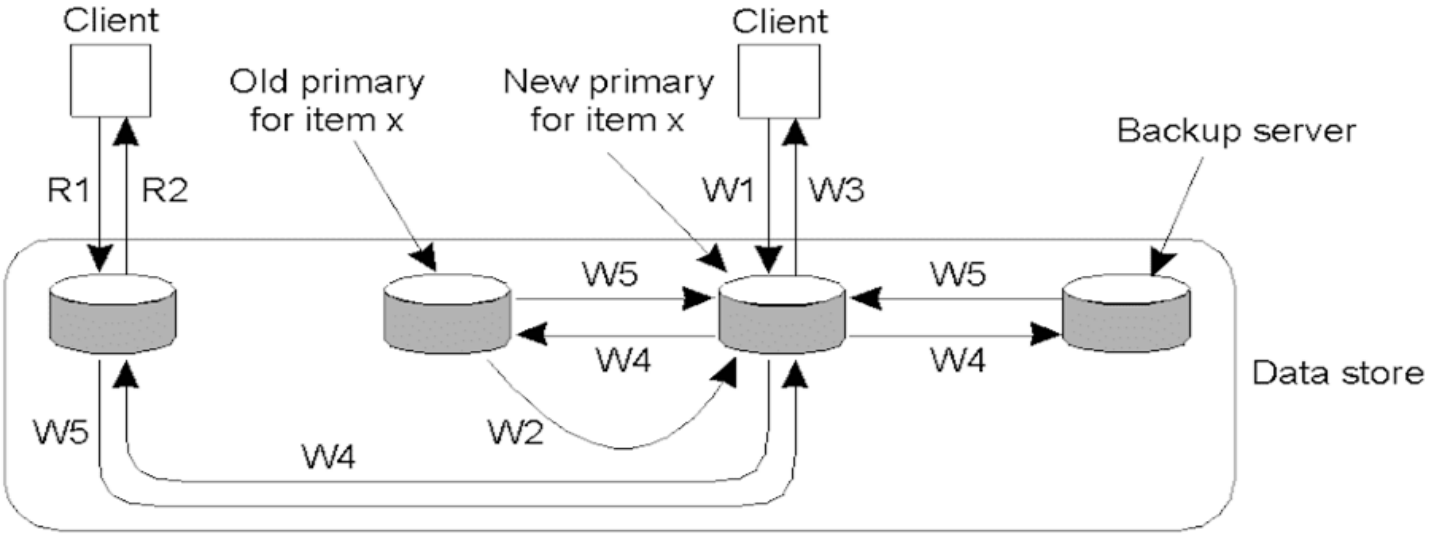
- Writes can take a long time (especially when a blocking write protocol is used)
- Alternative: Use a non-blocking write protocol
  - Primary acknowledges the client after updating its copy
  - If replicas failed during the update, read-your-writes consistency is not guaranteed

↑ Easy to implement sequential consistency

- Primary can send all writes to each replica in the same order via total+FIFO-order atomic multicast
- Implements strong consistency if read requests are also forwarded to the primary

Primary-backup local-write protocols

- Primary migrates to the replica that is writing, successive writes are carried out locally, and then the replicas are updated using a non-blocking protocol



W1. Write request
W2. Move item x to new primary
W3. Acknowledge write completed
W4. Tell backups to update
W5. Acknowledge update

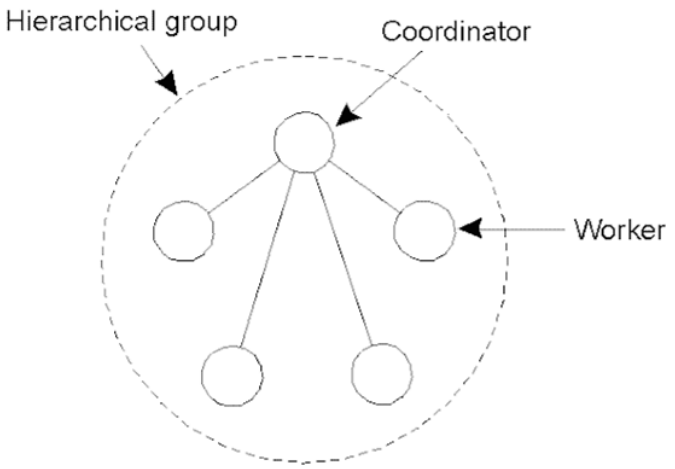
R1. Read request
R2. Response to read

Courtesy of [Tane06]

Hierarchical process groups

- Primary-based protocols to build hierarchical process groups for **fault tolerance**

  1. Clients send requests to the coordinator process (the primary)
     - Clients might be able to submit read requests to the workers
  2. Coordinator provides the desired service and updates the workers using total+FIFO-ordered atomic multicast



- If the primary fails, one of the workers is promoted to act as the primary
  - With crash/omission failures, a total of K + 1 processes are needed to survive K process failures
    - To provide service it is sufficient to have only one process up

Contents

- **Consistency protocols**
  - Primary-based protocols
  - **Replicated-write protocols**

Replicated-write protocols

- Writes can be carried out at **multiple replicas** instead of only one, as occurs in primary-based protocols
- Also called distributed-write protocols
- Two types:
  1. Active replication
  2. Quorum-based protocols (majority voting)

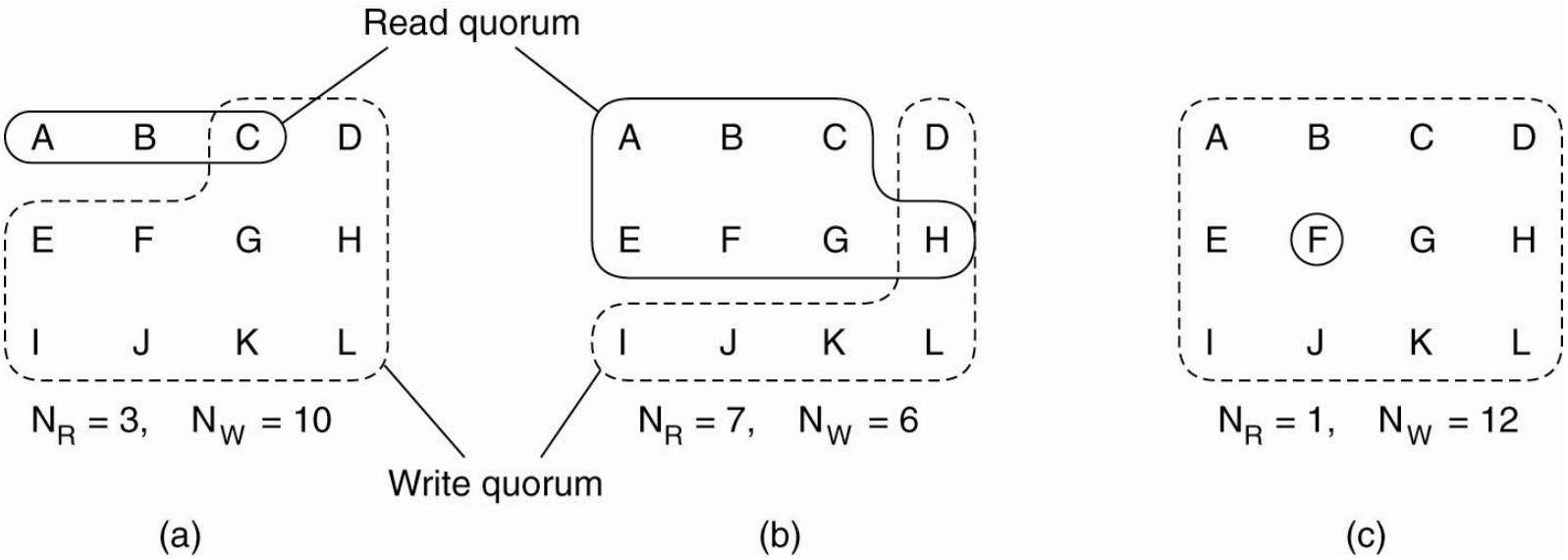
Active replication

- Each operation is forwarded to all replicas
  - Each replica has a process to carry out operations
- Operations need to be carried out in the same order everywhere
  - Client uses **total+FIFO-ordered multicast** to send operation to the group of replicas
  - Total+FIFO-ordered multicast ensures:
    - sequential consistency if we multicast only writes, and
    - strong consistency if we multicast also reads

Quorum-based protocols

- Clients must get permission from a quorum of the N replicas before either writing or reading
  - Write quorum of N_W replicas before writing
    - Update replicas in quorum, assign new version number
    - Remaining replicas are updated as a background task
  - Read quorum of N_R replicas before reading
    - Not all of the replicas in N_R need to be up-to-date
    - Any up-to-date replica may be read
  - $N_R + N_W > N$ to avoid read-write conflicts
  - $N_W > N/2$ to avoid write-write conflicts
  - Writes are serialized and reads return the latest version that was written → strong consistency

Quorum-based protocols



Courtesy of [Tane06]

a) Correct choice of read and write set
b) Choice that may lead to write-write conflicts
c) Correct choice, known as ROWA (read-one,write-all)

Quorum-based protocols

- How to configure N, W, and R depends on
  - what the common case is, and
  - which performance path needs to be optimized.
- R=1 and W=N

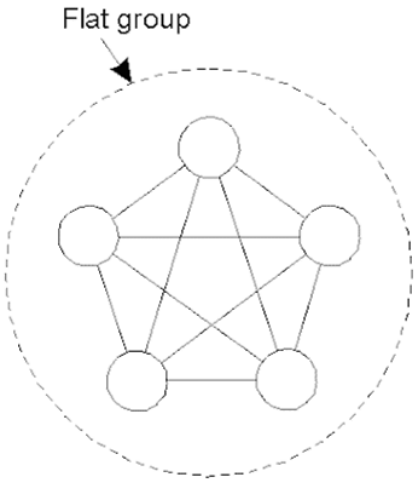
  →optimize for the read case
- W=1 and R=N

  → optimize for a very fast write
- W+R <= N

  → Weak/eventual consistency

Flat process groups

- Replicated-write protocols to build flat process groups for **fault tolerance**
  1. Clients send their requests to all replicas using total+FIFO-ordered atomic multicast
  2. Replicas process the request independently but identically and reply
     - All processes are identical



- If any replica crashes, the remaining ones continue to respond in the normal way
  A. With crash/omission failures, a total of K + 1 processes are needed to survive K process failures
     - If K members fail, the answer from the other is used

Flat process groups

B. With Byzantine failures, a total of 2K + 1 processes are needed to survive K process failures
   - Processes run even if sick (produce erroneous, random or malicious replies)
     - K failing processes could accidentally (or intentionally) generate the same reply
   - Response is defined through a voting mechanism, thus we need K+1 correct processes

- What if group members are not identical and non-faulty group members must reach agreement on the same value? This is a 'Consensus' problem

Summary

- In a WSC we need replication
  - Improve availability
  - Enhance reliability
  - Improve performance & scalability
- But replication can lead to inconsistencies …
- How can we propagate updates so that these inconsistencies are not noticed without severely degrading performance?
- Proposed solutions apply for the relaxation of any existing consistency constraints

Summary

- Consistency models
  - Data-centric models
    - Strong consistency concern itself with individual reads/writes to data items
    - Weaker models, which introduce the notion of synchronization variables, concern themselves with a group of reads/writes
  - Client-centric models
    - Concerned with maintaining consistency for a single client access to the distributed data store
    - Models based on Eventual consistency
  - Monotonic reads, Read your writes, Consistent Prefix, and Bounded Staleness

Summary

- We looked at various 'consistency protocols' which are specific implementations of consistency models
  - Primary-backup
    - Local-write & Remote-write
  - Replicated Write
    - Active replication & Quorum based
- The most common schemes were those that supported 'sequential consistency', but 'eventual consistency' is common now

Not only CAP: PACELC

- Extension to the CAP theorem.
- IF network partitioning (P), choose between
  - availability (A) and
  - consistency (C)
- Else (E), choose between
  - latency (L) and
  - loss of consistency (C)
- PA/EL
- PA/EC
- PC/EL
- PC/EC

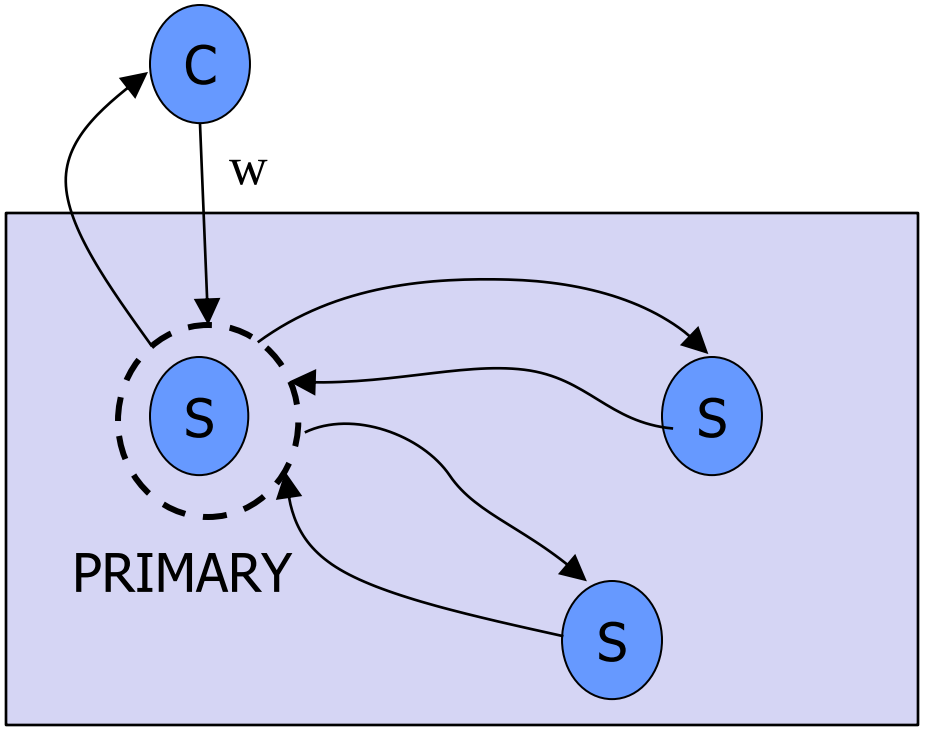
Abadi, Daniel J. "*Consistency Tradeoffs in Modern Distributed Database System Design*".2012

Contents

- Introduction
- History
- Eventual consistency
- **Examples**

Examples:

- N servers/replicas



- How do you enforce eventual consistency?
  - Primary backup remote-write

Examples:

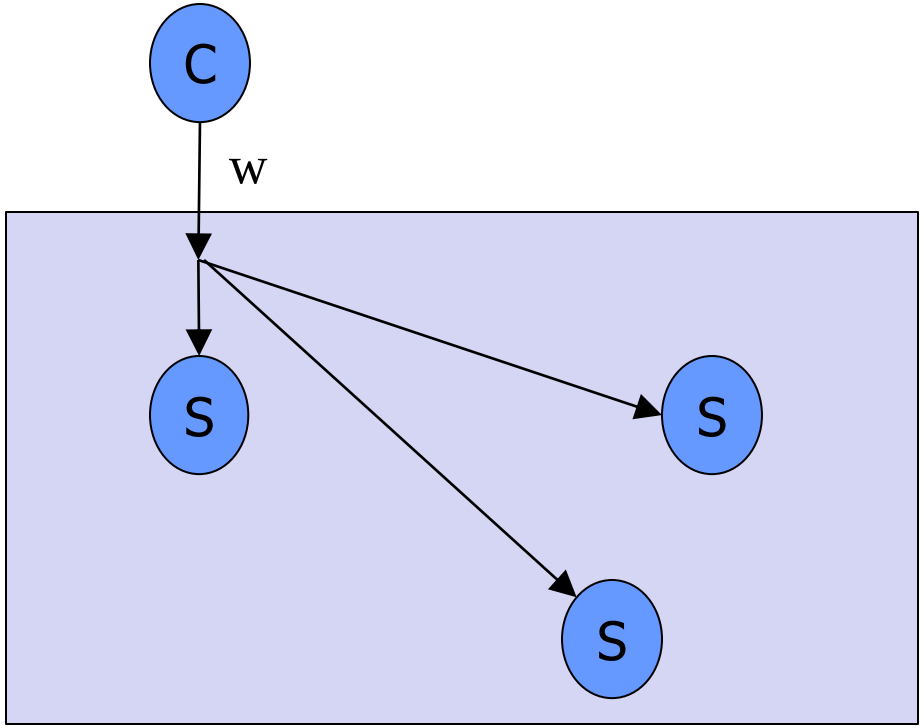
- How do you enforce strong consistency?
  - Primary backup remote-write
    - Read must go to the primary!
- The important thing to note here is that the architecture is the same, the implementation detail is what defines the consistency model!

Examples:

- N servers/replicas
- How do you optimize the case of a single client issuing consecutive writes to the same data item?
  - Primary backup local-write
- Oh, no! Multiple clients trying to write the same data item!
  - 1 primary is not enough!
    - Replicated-write

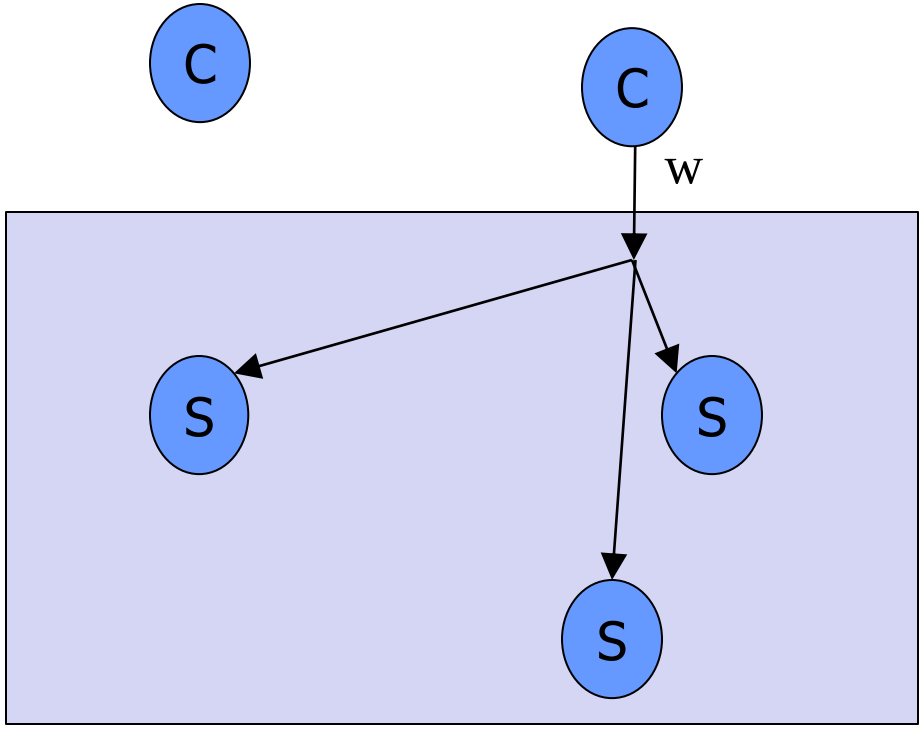
Examples:

- Forward write operations to all replicas



Examples:

- Forward write operations to all replicas



Examples:

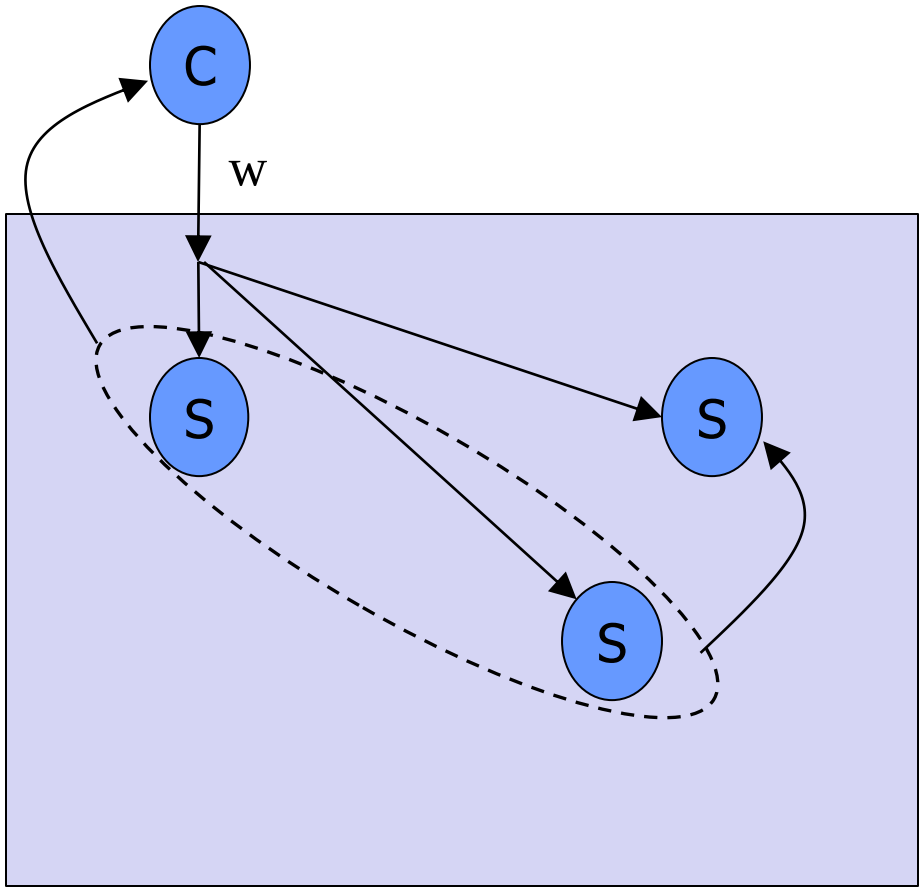
- Forward write operations to all replicas
  - What happens if 2 writes to same data at same time?
    - A TOTAL ORDER is needed → Sequencer ~ Coordinator
      - == Primary-backup

Examples:

- Quorum based
  - R + W > N → Avoid read-write conflicts
  - W > N/2→ Avoid write-write conflicts
  - R = 1 → Fast reads
  - W = 1 → Fast writes

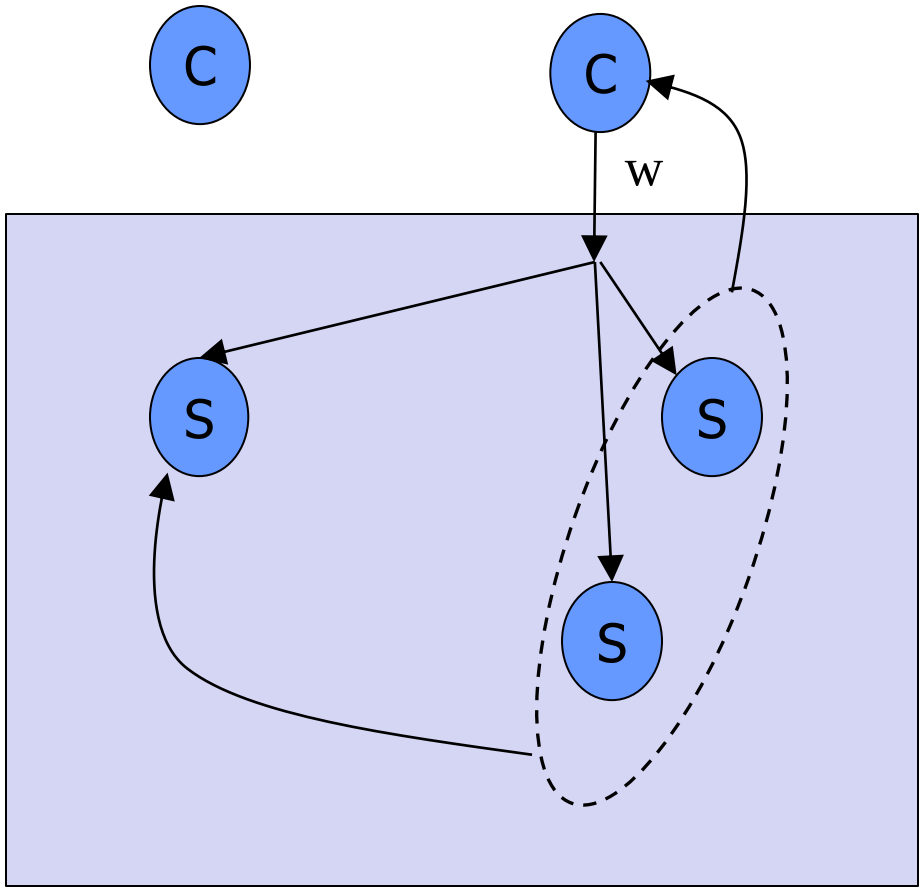
Quorum based protocol

W=2 R=2



Quorum based protocol

W=2 R=2



In-Memory DB

- The consistency model and the grouping of data made appear a lot of new technologies that hide the implementation of the infrastructure and eases the access to data
  - Memcached: key-value
  - Redis: key-value
  - CouchDB: Document based
  - MongoDB: Document based
- ACID vs BASE
  - **A**tomic, **C**onsistency, **I**solation, **D**urability
  - **B**asically **A**vailable, **S**oft state, **E**ventual consistency

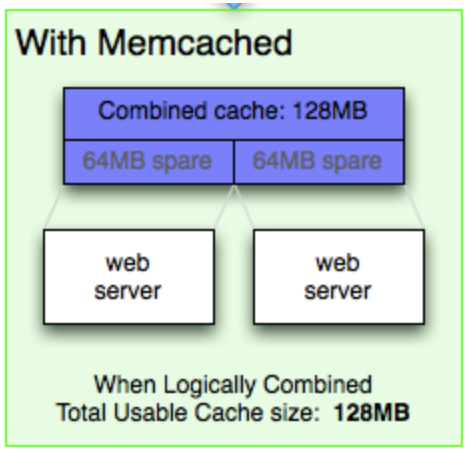
Examples

- **Memcached: key-value**
- Redis: key-value
- CouchDB: Document based
- MongoDB: Document based

Memcached

https://memcached.org/

- In-memory key-value store for small arbitrary data (strings, objects) from results of database calls, API calls, or page rendering.
- It is a client-server model
  - Client knows how to access the servers
  - Servers store the values
  - **No** communication between servers
    - Client-server only
  - LRU cache
    - Values expire after an specified amount of time



Memcached

- Example:

```
function get_foo(foo_id)
        foo = memcached_get("foo:" . foo_id)
        return foo if defined foo

        foo = fetch_foo_from_database(foo_id)
        memcached_set("foo:" . foo_id, foo)
        return foo
end
```

Memcached

- Consistent Hashing for keys
  - Model that allows for more stable distribution of keys given addition or removal of servers.
- Under the hood it uses a slab allocator
- Memcached memory → 1MiB/page
- Pages associated to slab classes
- Slab class → Fixed sized chunks
- #chunks/slab class ==

  #pages/sizeof(slabclass)

Examples

- Memcached: key-value
- **Redis: key-value**
- CouchDB: Document based
- MongoDB: Document based

Redis

http://redis.io/

- In-memory **data structure store**
  - Similar to memcached but with more data structures
    - Sets, Hashes, Sorted sets
    - Lists
    - Bitmaps
  - A publisher/subscriber messaging paradigm
  - Can be configured to run like memcached



Redis

http://redis.io/

- Master/replicas replication
  - Asynchronous
  - Non-blocking
- Allow writes only with N attached replicas
- Redis expires allow keys to have a limited time to live.

Examples

- Memcached: key-value
- Redis: key-value
- **CouchDB: Document based**
- MongoDB: Document based

CouchDB

https://couchdb.apache.org/

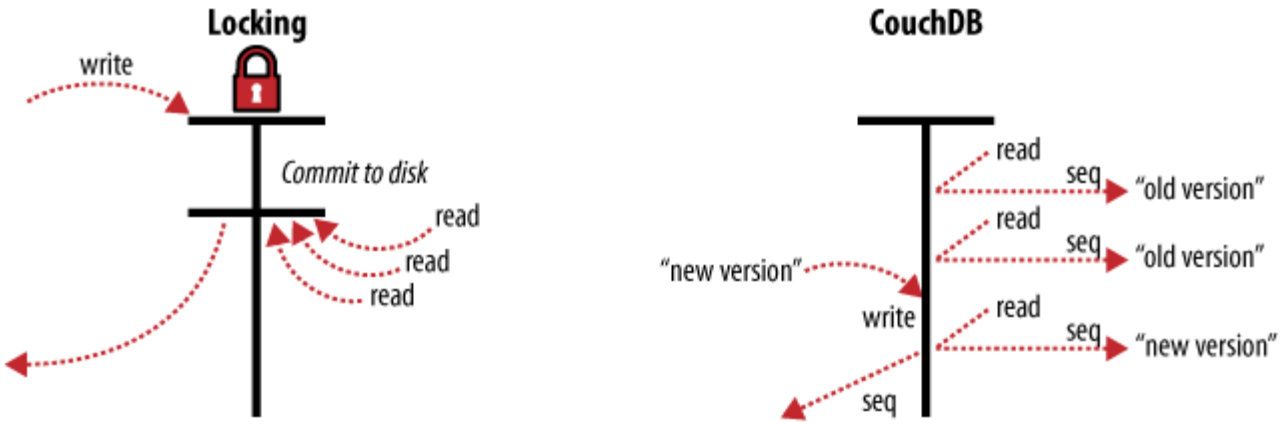
- Document based database
- Fault tolerant
- Easy scaling up/down
- Incremental replication
- Eventual consistency



CouchDB

https://couchdb.apache.org/

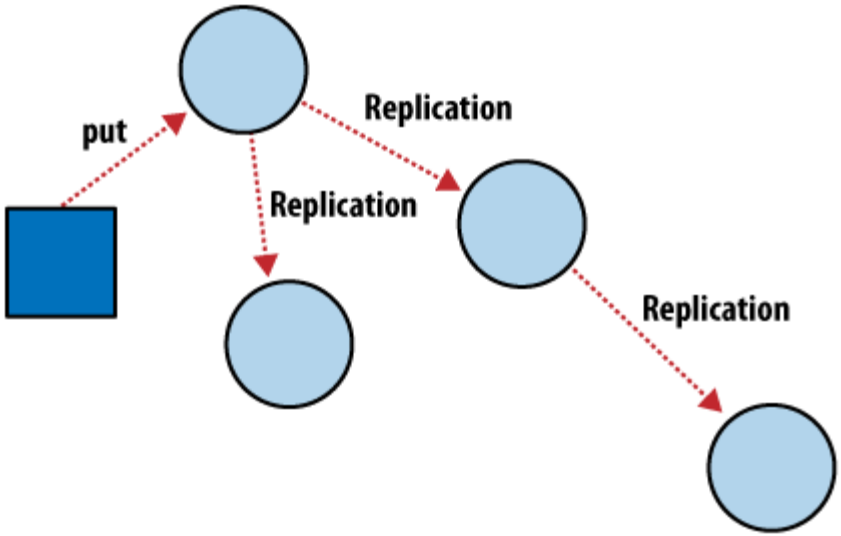
- Multi-Version Concurrency Model



CouchDB

https://couchdb.apache.org/

- Incremental Replication



CouchDB

- Cluster configuration
  - Q : Number of shards
    - Shard ~ part of database (→ 4 shards == 4 nodes)
  - N: Number of replicas (copies of every document)
  - R: Number of copies of a document with same revision to be read before returning it
  - W: Number of nodes needed to save the document

Examples

- Memcached: key-value
- Redis: key-value
- CouchDB: Document based
- **MongoDB: Document based**

MongoDB

https://www.mongodb.com/

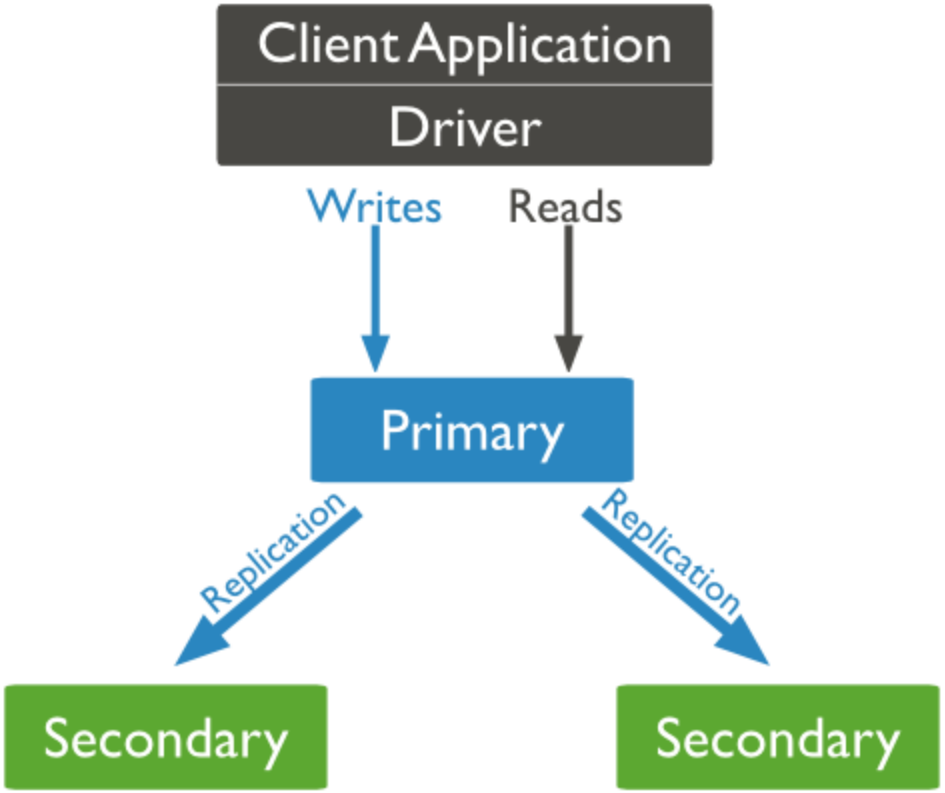
- MongoDB is an ~~open~~-source document database that provides high performance, high availability, and automatic scaling.
- Document based database
- Fault tolerant
- Easy scaling up/down
- Incremental replication
- Eventual consistency



MongoDB

https://www.mongodb.com/

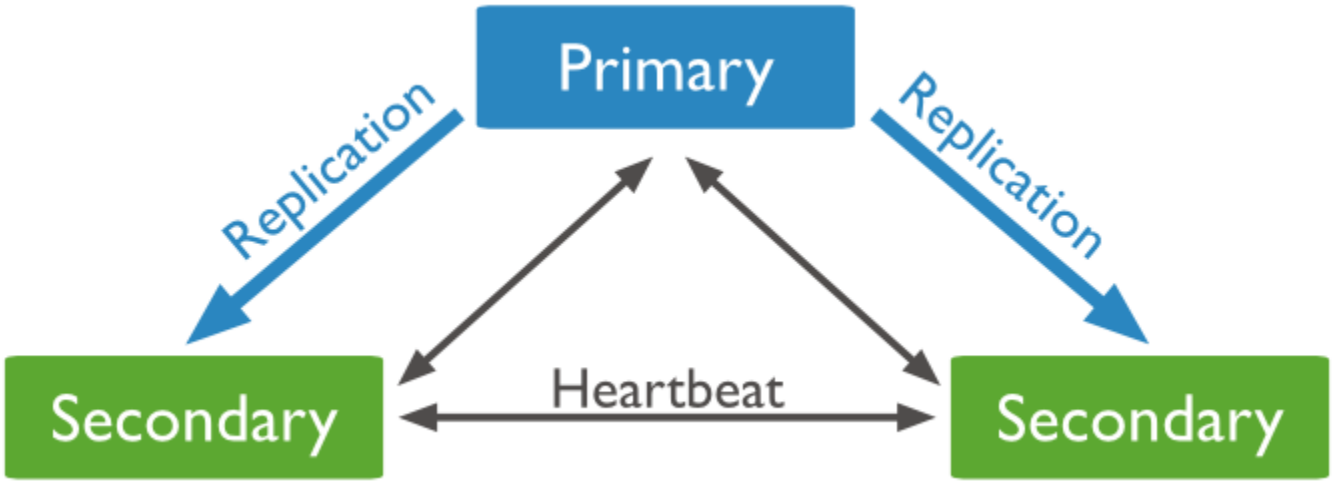
- Data replication



MongoDB

https://www.mongodb.com/

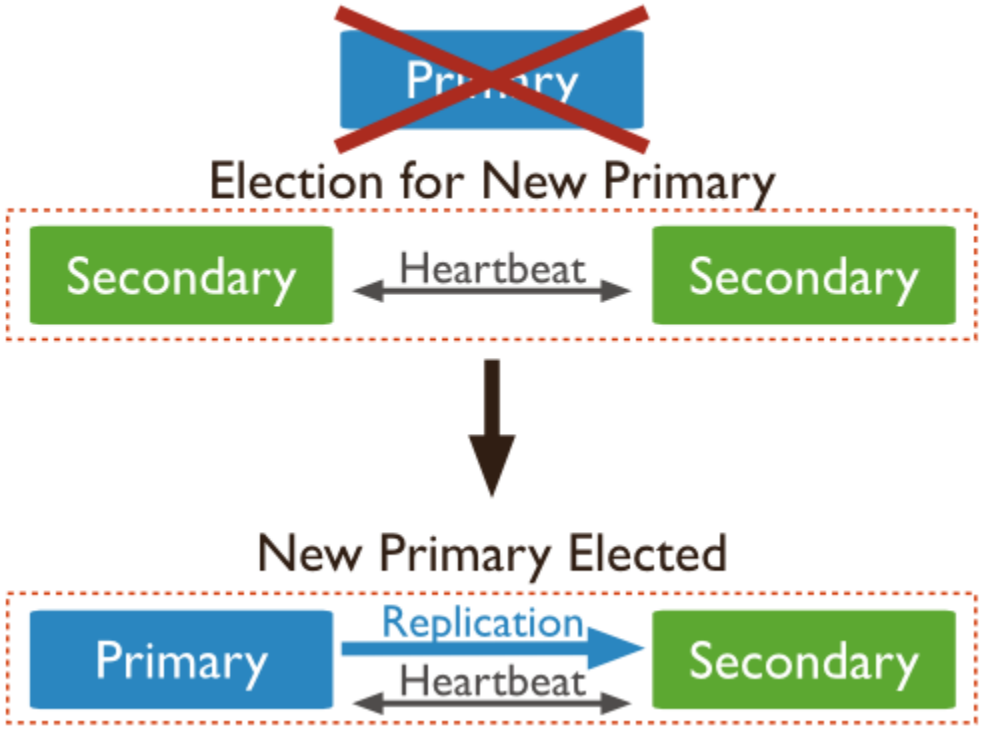
- **Asynchronous Replication**



MongoDB

https://www.mongodb.com/

- Automatic failover



Summary

- Different examples shown for:
  - Address spaces
    - Aggregated
    - Replicated
  - Consistency protocols:
    - Primary backup
    - Replicated write

References

[Duar08] Gustavo Duarte, *What Your Computer Does While You Wait*, 2008, http://duartes.org/gustavo/blog/post/what-your-computer-does-while-you-wait/

[Funk16] Mark Funk, *Drilling Into The CCIX Coherence Standard*, 2016, https://www.nextplatform.com/2016/07/13/drilling-ccix-coherence-standard/

[Jugo09] Jugones55, *MESI protocol diagram*, 2009, https://commons.wikimedia.org/w/index.php?curid=7136764

[Sand16B] Sandeep993, *State diagram for bus transactions*, 2016, https://commons.wikimedia.org/wiki/File:State_diagram_for_bus_transactions.png

[Sand16P] Sandeep993, *State diagram for processor transactions*, 2016, https://commons.wikimedia.org/wiki/File:State_diagram_for_processor_transactions.png

[Tane06] A. Tanembaum, *Distributed Systems, principles and paradigms*, 2006